

Course title:

Software Design Patterns (Lab)

Course code:

IS354

Problem 2:

Title : Implement a Singleton Pattern in Java

Problem Description: students must implement the class called database connection that act as a singleton class to provide connection to the clients to access the data from database.

Singleton Pattern ensures that a class has only one instance and provides a global access point to it.

Singleton means:

- Only **ONE object** of the class can exist.
- That object is **shared** by all clients.

Real-World Uses of Singleton Pattern

Used in:

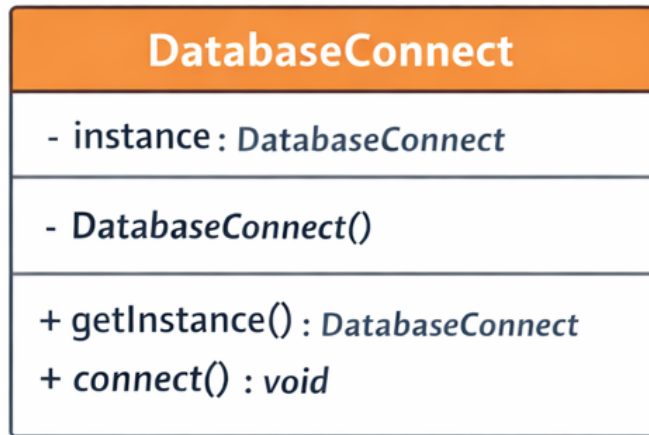
- ☐ Database connections
- ☐ Printer spooler
- ☐ Logger classes
- ☐ Configuration settings
- ☐ Cache management

Advantages of Singleton

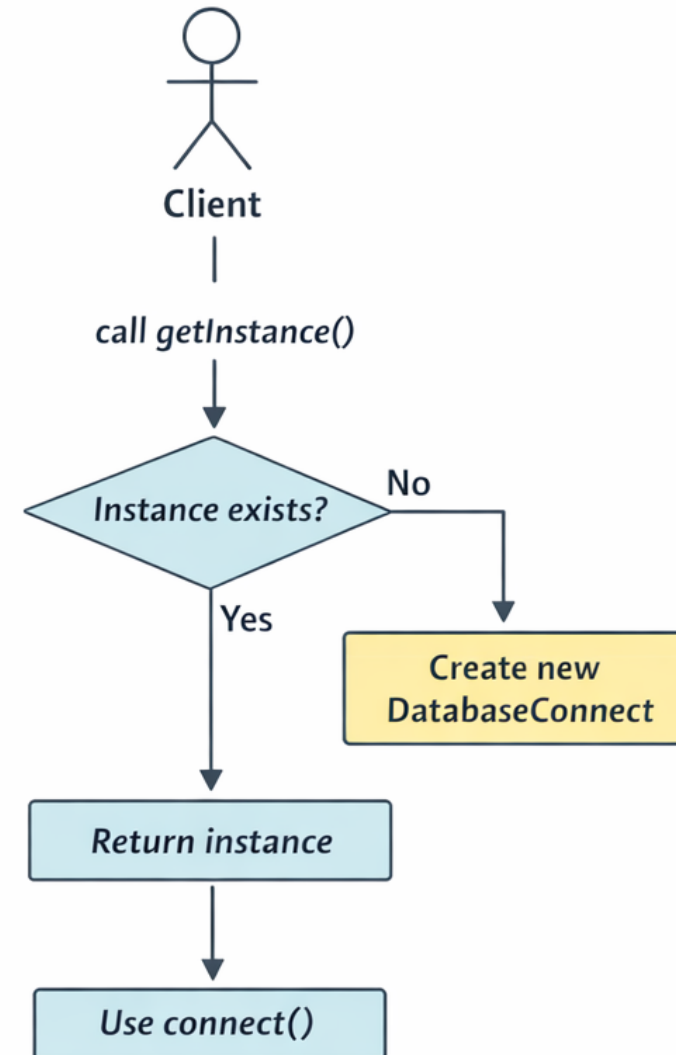
- ☐ Saves memory
- ☐ Controlled access to object
- ☐ Global point of access
- ☐ Prevents duplicate resources

Disadvantages

- ☐ Hard to test
- ☐ Can cause tight coupling
- ☐ Not thread-safe by default



UML Class Diagram of Singleton Pattern



Program Flow Diagram

```
public class DatabaseConnect {

    private static DatabaseConnect instance;

    private DatabaseConnect() {
        System.out.println("Database Connection Created...");
    }

    public static DatabaseConnect getInstance() {
        if (instance == null) {
            instance = new DatabaseConnect();
        }
        return instance;
    }

    public void connect() {
        System.out.println("Connected to Database...");
    }

    public static void main(String[] args) {

        DatabaseConnect db1 = DatabaseConnect.getInstance();
        db1.connect();

        DatabaseConnect db2 = DatabaseConnect.getInstance();
        db2.connect();

        if (db1 == db2) {
            System.out.println("Both objects are same — Singleton works!");
        }
    }
}
```

Singleton Pattern UML shows a class with a **private constructor**, a **static instance variable**, and a **public static method** to access the single object.

Program Flow Summary

Program starts



getInstance() called first time



Object created



Constructor runs



Same object returned every time